

Project Title: In-Browser QA Agent

Cost-Aware Self-Healing Browser Testing Assistant

Project Context

Project Summary:

The In-Browser QA Agent is a cost-aware, self-healing assistant for browser-based end-to-end testing. Testers describe supported workflows in constrained natural language; the system validates them as structured actions, executes them in Chromium with Playwright, and locates interface elements using DOM and accessibility evidence. When a target changes, the resolver escalates from exact and deterministic matching to an optional lightweight language model only when needed. Low-confidence or ambiguous repairs are rejected safely.

The MVP records the attempted resolution tier, candidate evidence, confidence, latency, model use, estimated cost, execution outcome, failure category, and rerun history. A dashboard makes this evidence visible and supports repeatable comparison of brittle, semantic, deterministic-healing, and hybrid AI-assisted approaches on a controlled mutation dataset.

Problem statement:

Browser tests built on IDs, CSS selectors, or XPath frequently fail after harmless interface changes such as renamed controls, added wrappers, reordered elements, or changed attributes. These false failures create locator-maintenance work and reduce trust in automation. AI-first repair can also introduce unsafe clicks, opaque decisions, latency, cost, and privacy risk. The project must therefore recover from minor locator drift only when evidence is sufficient, explain every non-exact decision, and distinguish locator failures from real regressions and flakiness.

Project goals:

1. Translate supported natural-language instructions into validated, allowed browser actions.
2. Execute common browser actions and assertions safely through Playwright in Chromium.
3. Recover from defined classes of locator drift through a cost-aware resolution ladder.
4. Measure confidence, abstain when evidence is weak, and prevent incorrect automatic healing.

5. Classify failures and identify controlled flaky behavior through repeatable reruns.
6. Expose audit evidence, run history, latency, AI usage, estimated cost, and basic accessibility/testability observations.
7. Benchmark four resolver modes on the same frozen scenarios and labeled interface mutations.

Application areas:

- End-to-end regression testing for frequently changing web interfaces.
- Self-healing locator research and QA automation education.
- Developer diagnosis of locator drift, functional regressions, and possible flakiness.
- Cost-aware AI experimentation using controlled, reproducible browser tasks.
- Basic feedback on missing accessible names, labels, roles, duplicate labels, and volatile identifiers.

Project justifications

Modern interfaces evolve faster than many test suites. Reducing false failures can save debugging time and improve confidence in automated quality gates. The proposed system is valuable because it combines accessibility-grounded evidence, deterministic matching, optional bounded AI reasoning, and mandatory abstention instead of relying on unrestricted autonomous control. It also produces measurable research evidence about recovery, false healing, latency, and AI-use trade-offs.

Limitations & Constraints:

- The MVP is Chromium-first and does not claim cross-browser equivalence.
- Only the documented constrained grammar and allowed actions are supported.
- Initial healing focuses on common buttons, links, fields, labels, headings, checkboxes, and basic form controls.
- Primary evaluation uses a controlled React application; claims beyond the labeled mutation classes are limited.
- Tier-3 model access is optional and adapter-based; multimodal reasoning is not required for MVP acceptance.
- Semantic equivalence can remain uncertain, so confidence thresholds and safe abstention are mandatory.
- Enterprise authentication, CI/CD integrations, mobile testing, and full visual regression are out of scope.

Assumptions:

- Target pages expose a standard browser DOM and run in a supported Chromium environment.
- Important controls provide some usable text, accessible name, label, role, attribute, or surrounding context.
- Minor UI evolution usually preserves the functional intent of the target element.
- Ground truth can be reviewed and frozen for controlled variants before final evaluation.
- Deterministic matching is preferred whenever it yields a unique high-confidence result.
- A small fixed rerun count provides useful evidence for controlled stable/flaky classification.

Team Member Details:

Name	Email ID	Github ID
Neev Badu	baduneev7@gmail.com	github.com/baduneev
Pankaj Kumar Mehta	123pankajmehta@gmail.com	github.com/pankajmehta07
Aradhya Dhungel	aradhyadhungelr@gmail.com	github.com/uwantwater

Requirements

Expected Inputs:

- Constrained natural-language scenarios containing supported browser actions and assertions.
- Target URL or selected version of the controlled React test application.
- Execution configuration: rerun count, enabled resolution tiers, confidence policy, and AI-escalation setting.
- DOM and accessibility evidence including role, accessible name, visible text, labels, attributes, and surrounding context.
- Previously verified target evidence, when available.
- Versioned mutation ground truth and stable/flaky labels for evaluation.

Expected outputs:

- Validated structured actions or actionable validation errors before browser execution.
- Per-step execution status, timing, browser errors, and target evidence.
- An accepted self-healing decision or safe abstention with tier, candidates, confidence, latency, and AI-use evidence.
- A plain-language failure category and explanation that separates healing outcome from functional assertion outcome.
- Persistent run history, rerun sequence, flaky classification, estimated cost, and accessibility/testability observations.
- JSON or CSV experiment outputs and summary metrics for baseline comparison.

End-User & Stakeholders Requirements

The following user stories define the tasks and outcomes required by testers, developers, project leads, researchers, and evaluators.

Requirements	Details	Priority	Success Criteria
QA engineer — author tests	As a QA engineer, I want to create supported multi-step tests in constrained natural language so that I can automate common browser workflows without writing complete Playwright scripts.	Must	At least 90% of frozen supported instructions parse correctly and executable steps are previewed.
Tester — heal locator drift	As a tester, I want minor locator changes to be repaired only when evidence is strong so that false failures decrease without unsafe actions.	Must	Hybrid healing succeeds on over 50% of recoverable cases and false healing remains below 10%.

Requirements	Details	Priority	Success Criteria
Developer — diagnose failures	As a developer, I want a clear root-cause explanation and evidence so that I can distinguish locator drift, ambiguity, regression, navigation error, assertion failure, and flakiness.	Must	Every failed step shows a category and supporting evidence; healing and assertion outcomes remain separate.
Team lead — assess cost	As a team lead, I want tier, latency, and model-cost reporting so that I can judge whether the hybrid strategy is operationally practical.	Must	More than half of routine resolution attempts complete without a model call; per-tier latency is reported.
Researcher — compare baselines	As a researcher, I want repeatable experiments across four resolver modes so that system claims are measurable and reproducible.	Must	The same frozen cases run in all modes and results export overall and by mutation category.

Functional Requirements

“Shall” denotes an independently verifiable requirement. Priorities use MoSCoW: Must, Should, Could, and Won’t for the MVP.

Requirements	Details	Priority	Success Criteria
Test definition and configuration			
FR-001 — Natural-language tests	The system shall accept multi-step tests written in the documented constrained natural-language grammar.	Must	Passes the frozen parsing test set and demonstration.
FR-002 — Structured actions	Each supported instruction shall become a JSON action containing an action type, target, and optional value or assertion.	Must	All supported cases validate against the versioned action schema.
FR-003 — Actionable rejection	Unsupported, incomplete, or schema-invalid instructions shall be rejected before browser execution with field- or step-level feedback.	Must	Negative cases execute no browser action and show a corrective message.
FR-004 — Target selection	The dashboard shall accept a target URL or a selected local application and DOM variant.	Must	UI test confirms both selection paths and input validation.

Requirements	Details	Priority	Success Criteria
FR-005 — Run configuration	The dashboard shall expose rerun count, confidence policy, allowed healing strategies, and AI-escalation settings.	Must	Saved run configuration matches the values shown in the UI.
FR-006 — Action preview	The system should display the generated action sequence and validation result before or at execution start.	Should	UI acceptance test shows ordered actions and validation state.
Browser execution			
FR-007 — Chromium control	The runner shall launch and control Chromium through Playwright.	Must	Baseline end-to-end journey completes in Chromium.
FR-008 — Supported operations	The runner shall support goto, click, type, check, select, assertVisible, assertText, and assertContains.	Must	Each operation passes its dedicated end-to-end fixture.
FR-009 — Allowlisted execution	The runner shall execute only schema-approved actions and shall not run arbitrary model-generated source code.	Must	Injection and invalid-output suites produce no unauthorized action.
FR-010 — Step telemetry	The runner shall capture step start/end time, outcome, and browser error information.	Must	Every executed step has complete timing and outcome fields.
FR-011 — Failure policy	The runner shall stop or continue after a failed step according to an explicit configured policy.	Must	Both policies produce the documented ordered outcomes.
FR-012 — Deterministic replay	The system should replay a stored test and configuration against a selected page variant.	Should	Replay preserves test/configuration references and ordered steps.
Semantic extraction and candidate generation			
FR-013 — Semantic evidence	The extractor shall capture applicable role, accessible name, visible text, label, tag, ID, name, and placeholder.	Must	Extractor fixtures contain expected evidence for supported controls.
FR-014 — Context evidence	The extractor shall capture sufficient parent, child, or surrounding-text context for context-aware ranking.	Must	Structural-mutation fixtures retain differentiating context.

Requirements	Details	Priority	Success Criteria
FR-015 — Candidate representation	Candidate records shall distinguish missing evidence from empty values and identify the page and run context.	Must	Schema tests validate null/empty handling and correlation fields.
FR-016 — Bounded candidates	Candidate generation shall be limited to relevant interactive or assertion-compatible elements before Tier-3 inference.	Must	Candidate-set inspection finds no unrelated executable targets.
FR-017 — Optional geometry	The system should capture bounding information only when an enabled strategy or audit view requires it.	Could	Configuration test confirms geometry collection can be disabled.
Cost-aware resolution and self-healing			
FR-018 — Cost-ordered tiers	The resolver shall attempt enabled tiers in ascending cost order and record every attempted tier.	Must	Resolver traces show ordered attempts without skipped mandatory tiers.
FR-019 — Verified locator	Tier 0 shall accept a previous locator only when it resolves and stored semantic evidence remains consistent.	Must	Changed-target fixtures reject stale or semantically inconsistent locators.
FR-020 — Exact semantics	Tier 1 shall resolve unique exact matches using role/name, label, placeholder, or visible text as applicable.	Must	Exact-match fixtures select the single correct target.
FR-021 — Deterministic similarity	Tier 2 shall compute configurable similarity using role, text, label, context, and attribute evidence.	Must	Scoring unit tests reproduce expected candidate order.
FR-022 — Candidate ranking	The resolver shall rank plausible candidates and retain their scores and evidence for audit.	Must	Every non-exact decision stores the reviewed ranking.
FR-023 — Conditional model use	Tier 3 shall run only when enabled and deterministic tiers cannot produce an accepted decision.	Must	Routing tests confirm no unnecessary model calls.
FR-024 — Bounded model context	Tier 3 shall receive target intent, structured candidates, and relevant previous-target context, not unrestricted executable page code.	Must	Adapter payload contains only allowed, redacted fields.

Requirements	Details	Priority	Success Criteria
FR-025 — Model validation	Model output shall pass a strict response schema and candidate-ID validation before use.	Must	Malformed or unknown-candidate responses are rejected safely.
FR-026 — Confidence	The resolver shall calculate normalized confidence and identify the policy and threshold used.	Must	Calibration and log tests record values in the valid range.
FR-027 — Mandatory abstention	Automatic healing shall be rejected when confidence is below policy or candidates remain materially ambiguous.	Must	All labeled low-confidence or ambiguous fixtures abstain.
FR-028 — Safe rejection	A rejected heal shall not execute the target action and shall return an explicit safe failure.	Must	Safety end-to-end test detects zero post-rejection target actions.
FR-029 — Healing audit	An accepted heal shall record expected and resolved target, tier, candidates, confidence, latency, AI use, and outcome.	Must	Audit completeness is 100% for accepted non-exact heals.
FR-030 — Multimodal scope	Tier-4 multimodal reasoning may be an isolated experiment and shall not be required for core MVP operation.	Won't	MVP passes all acceptance gates with Tier 4 disabled.
Failure intelligence and flakiness			
FR-031 — Failure categories	The system shall categorize supported failures as locator drift, ambiguous target, missing element, assertion, navigation, or possible flakiness when evidence supports the label.	Must	Labeled failure fixtures match the expected categories.
FR-032 — Failure explanation	Each failed action shall include a plain-language explanation and supporting evidence.	Must	Failure detail view contains both elements for every failed action.
FR-033 — Controlled reruns	When enabled, the system shall rerun the same test/configuration the configured number of times and preserve ordered outcomes.	Must	Stored rerun count and sequence match the request.

Requirements	Details	Priority	Success Criteria
FR-034 — Flaky classification	All-pass shall be stable pass, all-fail stable fail, and mixed outcomes flaky.	Must	Synthetic sequences produce the expected classification.
FR-035 — Flaky evidence	The system should display rerun count and pass ratio with the flaky classification.	Should	Dashboard shows count, ordered outcomes, ratio, and label.
FR-036 — Outcome separation	The system shall keep locator-healing outcome separate from the final functional assertion outcome.	Must	End-to-end results expose both fields independently.
Persistence, dashboard, and audit			
FR-037 — Run record	The system shall persist run ID, timestamp, test/configuration reference, status, duration, healing count, AI calls, estimated cost, and flaky flag.	Must	Completed run records contain all required fields after restart.
FR-038 — Linked evidence	Step results and non-exact candidate/decision evidence shall be linked to the parent run.	Must	Relational or JSON integrity tests find no orphan records.
FR-039 — Execution dashboard	The dashboard shall show run and step results, original/resolved target, tier, confidence, latency, cost, root cause, and flaky status.	Must	UI acceptance test verifies each field for representative runs.
FR-040 — Run history	The dashboard should provide searchable or filterable run history.	Should	History test filters by at least status, date, or test reference.
FR-041 — Cost estimate	The system should estimate model-use cost with a documented pricing or local-cost policy and identify the estimate basis.	Should	Calculation test matches the configured policy and records its version.
FR-042 — Accessibility observations	The system should report missing names/labels/roles, ambiguous duplicate labels, and volatile IDs as basic observations.	Should	Known audit fixtures generate the expected observations.
FR-043 — Export	The system should export or display experiment records and summary metrics in JSON and/or CSV.	Should	Export validates against its schema and matches stored totals.

Requirements	Details	Priority	Success Criteria
Controlled evaluation support			
FR-044 — Mutation application	The controlled application shall include stable pages and labeled variants for identifier, class, text, wrapper, sibling, order, movement, attribute, ambiguity, content-only, and functional-failure changes.	Must	Dataset inventory contains every named mutation class.
FR-045 — Ground truth	Each case shall record target existence, whether healing is permitted, the correct evolved target, and expected abstention.	Must	Manual ground-truth audit finds all required fields.
FR-046 — Evaluation modes	Evaluation shall support brittle locator, Playwright semantic locator, deterministic self-healing, and hybrid SLM-assisted modes.	Must	Experiment configuration executes all four modes.
FR-047 — Experiment record	Every experiment shall record mutation category, strategy, candidates, decision, confidence, correctness, latency, and AI usage.	Must	Dataset completeness check reports 100% required fields.
FR-048 — Segmented results	Results shall be reportable overall and by mutation category and resolution tier.	Must	Analysis output contains all required aggregations.

Non-Functional requirements

These requirements define how well the product must perform in accuracy, safety, cost, reliability, security, privacy, usability, maintainability, portability, observability, and accessibility.

Requirements	Details	Priority	Success Criteria
NFR-001 — Accuracy	Supported instruction parsing shall achieve at least 90% correctness on the frozen evaluation set.	Must	Instruction Success Rate ≥ 0.90 .
NFR-002 — Safety	Incorrect accepted heals shall remain below 10%; below 5% is the stretch target.	Must	False-Healing Rate < 0.10 ; stretch < 0.05 .

Requirements	Details	Priority	Success Criteria
NFR-003 — Healing effectiveness	The hybrid resolver shall correctly recover more than half of labeled recoverable broken targets.	Must	Healing Success Rate > 0.50, reported by mutation.
NFR-004 — Cost-awareness	More than half of routine resolution attempts shall complete without a model call.	Must	AI escalation rate < 0.50.
NFR-005 — Performance	Tiers 0–2 shall require no network AI call and resolution latency shall be measured separately by tier.	Must	Trace confirms zero AI calls at T0–T2 and reports tier latency.
NFR-006 — Reliability	Fixed inputs and configuration shall produce reproducible logs aside from documented nondeterministic sources.	Must	Replay comparison matches required deterministic fields.
NFR-007 — Durability	An interrupted run shall not corrupt previously completed run history.	Must	Fault-injection test preserves all completed records.
NFR-008 — Traceability	Every non-exact decision shall identify candidates, selected/rejected outcome, evidence, confidence, tier, and AI use.	Must	Audit completeness = 100%.
NFR-009 — Security	Natural-language and model outputs shall be constrained by allowlisted schemas and values before execution.	Must	Injection and invalid-output tests execute no unauthorized action.
NFR-010 — Privacy	Sensitive inputs shall be redacted from user-visible and persistent logs unless explicitly required for testing.	Must	Zero plaintext configured secrets in the redaction suite.
NFR-011 — Usability	Users shall be able to understand what ran, what was selected, whether healing occurred, and why a failure occurred.	Must	Scenario review completes without raw-log inspection.
NFR-012 — Maintainability	Parser, runner, extractor, resolver, model adapter, analyzers, persistence, and dashboard shall remain separately testable modules.	Must	Architecture and code review confirms modular test boundaries.
NFR-013 — Portability	The MVP should run on documented environments with Node.js, npm, and Chromium dependencies installed.	Should	Clean-environment setup and smoke test pass.

Requirements	Details	Priority	Success Criteria
NFR-014 — Observability	Errors should include run and step correlation identifiers without exposing secrets.	Should	Log review finds correlation IDs and no configured secrets.
NFR-015 — Accessibility	Dashboard controls and result states should use semantic roles, labels, and keyboard-accessible interaction.	Should	Automated checks plus keyboard review find no critical blocker.

Data collection: Required data / source of the data

Data identification:

The project needs qualitative and quantitative data. Qualitative data includes natural-language test intent, semantic element descriptions, failure explanations, and accessibility/testability observations. Quantitative data includes candidate scores, confidence, correctness labels, pass/fail sequences, latency, model usage, estimated cost, and aggregate performance metrics. The authoritative research dataset is a versioned collection of baseline scenarios, labeled interface mutations, expected targets, expected abstentions, and controlled stable/flaky variants.

Data Sources:

- A team-built controlled React application with stable baseline pages and labeled UI variants.
- Playwright browser events and DOM/accessibility evidence captured during execution.
- Manually reviewed ground truth for intended targets, permitted repairs, expected abstention, and failure category.
- Structured, redacted model-adapter requests and validated responses when Tier 3 is enabled.
- Optional public demonstration pages for supplementary validation only; the controlled local dataset remains authoritative.

Data Acquisition methods:

1. Define baseline user journeys and expected targets in the controlled application.
2. Create and version one labeled mutation at a time across identifier, text, attribute, structure, ambiguity, content-only, and real-regression categories.
3. Run identical scenarios with Playwright under fixed configurations and collect browser, DOM, accessibility, resolver, and timing evidence.
4. Record candidates, decisions, confidence, correctness, execution outcome, AI use, estimated cost, and correlated run/step identifiers.

5. Generate stable and intentionally nondeterministic variants, rerun them under fixed settings, and preserve ordered outcomes.
6. Review ground truth manually and freeze the final evaluation set before final tuning.

Data collection challenges:

- Ambiguous semantic equivalence may leave multiple plausible controls
- Ground-truth bias may favor the designed resolver
- Controlled pages may not represent all production applications
- Timing, network, and application state can contaminate flaky labels
- Typed credentials or tokens could enter evidence.
- External pages may change unexpectedly

Benchmarking the Model:

Establishing a baseline model:

All approaches will run against the same frozen scenarios, mutations, ground truth, and runtime configuration. Results will be reported overall and by mutation category, resolution tier, and accepted/rejected outcome.

- B0 — Brittle locator: ID/CSS/XPath-like targeting without healing; minimum reference for locator drift.
- B1 — Playwright semantic locator: role, accessible name, label, placeholder, or visible-text matching without custom healing.
- B2 — Deterministic self-healing: exact semantic matching plus similarity ranking and abstention, with no model inference.
- B3 — Hybrid SLM-assisted resolver: B2 plus bounded Tier-3 reasoning; the primary proposed system.

Primary metrics: Instruction Success Rate, Healing Success Rate, False-Healing Rate, healing precision/recall/F1, abstention rate, AI-escalation rate, per-tier resolution latency, estimated cost per run, flaky-classification accuracy, root-cause accuracy, and audit completeness.

Resources required:

- Development environment: TypeScript, Node.js, React, Playwright, Chromium, npm, Git, and a shared GitHub repository for version control and collaboration.
- AI services: Access to a lightweight language model through a local runtime or API for optional Tier-3 resolution. Limited API credits may be required for experiments
- Test application and dataset: A controlled React application containing baseline pages, labeled UI mutations, expected targets, failure cases, and stable/flaky test scenarios.

- Data storage and analysis: Local JSON/CSV files or a lightweight database for storing test runs, candidate evidence, confidence scores, latency, model usage, and evaluation results.
- Testing tools: Unit, integration, and end-to-end testing frameworks, Playwright test fixtures, schema-validation tools, and basic accessibility-testing utilities.
- Computing resources: Standard developer laptops capable of running the React application, Node.js services, and Chromium. A dedicated GPU is optional and only needed if the language model is executed locally.